EXERCICE 1 : LES FONCTIONS

Exercice 1. Ecrire la fonction listeVehicule(NomAg ....) qui retourne la liste des véhicules proposées (colonne exemple) et le prix journalier pour chaque véhicule, pour une agence passée en paramètre (nom de l'agence). Message d'erreur si l'agence n'existe pas.

Vous utiliserez les valeurs suivantes pour vos tests : ('Micsot') / ('Ade') / ('Micot')

```
CREATE OR REPLACE FUNCTION listeVehicule(p_nom_ag varchar(45)) RETURNS setof record AS
$$
DECLARE
    v_ligne record ;
BEGIN
    -- on vérifie que l'agence existe
    -- perform est comme le select mais ne sert qu'à évaluer une requête
    perform nom_agence from agence  where lower(nom_agence) =lower(p_nom_ag);
    -- si aucune ligne retournée
    IF not found
          THEN RAISE EXCEPTION 'l''agence % n''existe pas', p_nom_ag;
    END IF;

  -- Requête qui retourne les véhicules proposés par l'agence passée en paramètre
    FOR v_ligne IN select exemple, prix_jour
                from vehicule inner join proposer using(code_vehicule)
                              inner join agence  using(num_agence)
                              where lower(nom_agence) =lower(p_nom_ag)
    LOOP
          RETURN NEXT v_ligne;
    END LOOP;

    RETURN;
END
$$ LANGUAGE plpgsql;

-- jeux de test
SELECT * FROM listeVehicule('Micsot') AS (nom_vehi varchar, px numeric)  ;
```

	A-Z nom_vehi	123 px
1	peugeot partner	131
2	citroen C5	127
3	peugeot 107	51
4	ford transit	128

```
SELECT * FROM listeVehicule('Ade') AS (nom_vehi varchar, px numeric)  ;
```

	A-Z nom_vehi	123 px
1	renault mégane 1,6	98
2	renaul kangoo	107
3	peugeot partner	131
4	ford transit	128
5	citroen C5	127
6	toyota yaris	78
7	citroen jumper	145
8	opel corsa	33
9	citroen C4	119
10	TOYOTA PRIUS	103

```
SELECT * from listeVehicule('Micot') as (nom_vehi varchar, px numeric)  ;
                -- ERREUR: l'agence Micot n'existe pas
```

Exercice 2. Ecrire la fonction SommePaye(NumeroClient….) qui retourne pour un client passé en paramètre (numéro du client) le cumul des sommes qu'il a payé. Le calcul se base sur le nombre de jours de location par réservation **multiplié** par le prix du véhicule loué.

Message d'erreur si le client n'existe pas.

Vous utiliserez les valeurs suivantes pour vos tests : (100) / (102) / (103) / (33)

```
CREATE OR REPLACE FUNCTION SommePaye(p_numCli int) RETURNS real AS $$
DECLARE
    somme real;
BEGIN
      -- on vérifie que le client existe
      PERFORM num_client FROM client WHERE num_client = p_numCli;
      -- si aucune ligne retournée
      IF NOT FOUND
            THEN RAISE EXCEPTION 'le client % n''existe pas', p_numCli;
      END IF;

   -- Requête qui retourne les véhicules proposés par l'agence passée en paramètre
      SELECT sum(prix_jour * extract(day from(date_heure_retour-date_heure_depart)))
      INTO somme
      FROM reservation inner join vehicule using(code_vehicule)
      WHERE num_client= p_numCli;

   RETURN somme;
END
$$ LANGUAGE plpgsql;

-- jeux de test
 select SommePaye(100);   -- 5129
 select SommePaye(102);   -- 7402
 select SommePaye(103);   -- 1016
 select SommePaye(33);    -- ERREUR: le client 33 n'existe pas
```

Exercice 3. Ecrire la fonction nb_pts (NumeroClient .., Num_vehi.., date_deb .., date_fin ..) qui retourne le nombre de points déjà acquis, le nombre de points attribuer et le nombre total de point (déjà acquis + points attribuer) pour un client donné, un véhicule donné et une date de début et de fin de réservation.

La fonction vérifiera que :

- Le client existe et qu'il est Fidèle, erreur sinon
- Que le véhicule existe, erreur sinon
- Que la date de début et plus petit de celle de fin, erreur sinon

```
CREATE OR REPLACE FUNCTION nb_pts(NumeroClient int, Num_vehi int, date_deb date,
date_fin date)  RETURNS record AS $$
DECLARE
    info record  ;
BEGIN
       -- on vérifie que le client existe
       perform num_client from client where num_client = NumeroClient;
       IF not found
             THEN RAISE EXCEPTION 'le client % n''existe pas', NumeroClient;
       END IF;

       -- on vérifie que le client est Fidèle
       perform num_client from client  where type_client='Fidéle' and num_client =
NumeroClient  ;
       IF not found
             THEN RAISE EXCEPTION 'le client % n''est pas Fidéle', NumeroClient;
       END IF;

       -- on vérifie que le véhicule existe
       perform code_vehicule from vehicule where code_vehicule = Num_vehi;
       IF not found
             THEN RAISE EXCEPTION 'le véhicule % n''existe pas', Num_vehi;
       END IF;

       -- on vérifie que les dates sont cohérentes
       IF (date_deb > date_fin)
             THEN RAISE EXCEPTION 'la date de début ne peut être plus grande que la
date de fin';
       END IF;

  -- Requête qui retourne le nombre de point
       select coalesce(pts_bonus_acquis,0) as pts_deja_acquis,
                              (select nb_points from attribuer
                                    where nb_jours <= date_fin-date_deb
                                    and code_vehicule = Num_vehi
                                    order by nb_jours desc
                                    limit 1) as nb_pts_attribuer,
                              coalesce(pts_bonus_acquis,0) + (select nb_points ²
                                    from attribuer
                                    where nb_jours <= date_fin-date_deb
                                    and code_vehicule = Num_vehi
                                    order by nb_jours desc
                                    limit 1) as pts_total
       into info
       from client where num_client=NumeroClient ;

   RETURN info;
end;
$$ LANGUAGE plpgsql;

-- jeux de test
```

```
-- jeux de test
select * from nb_pts(300,1, '2020-06-30', '2020-06-20')
      as (pts_déja_acquis  int, pts_attribuer int, pts_total int);
                  -- ERREUR: le client 300 n'existe pas
select * from nb_pts(100,78, '2020-06-30', '2020-06-20')
      as (pts_déja_acquis  int, pts_attribuer int, pts_total int);
                  -- ERREUR: le véhicule 78 n'existe pas
select * from nb_pts(103,1, '2020-06-20', '2020-06-30')
      as (pts_déja_acquis  int, pts_attribuer int, pts_total int);
                  -- ERREUR: le client 103 n'est pas Fidéle
select * from nb_pts(104,1, '2020-06-30', '2020-06-20')
      as (pts_déja_acquis  int, pts_attribuer int, pts_total int);
                  -- ERREUR: la date de début ne peut être plus grande que la date de fin
select * from nb_pts(104,1, '2020-06-20', '2020-06-30')
      as (pts_déja_acquis  int, pts_attribuer int, pts_total int);
                  -- la date de début ne peut être plus grande que la date de fin
```